

Посібник з розгортання

Запустіть навчену наглядову мережу як тіньовий/рекомендаційний worker над PLC/PID/SIS

Половина конвеєра, що відповідає за розгортання: візьміть мережу JNeopallium, яку створив тренер, — разом із готовим промисловим контекстом — під'єднайте її до ваших потоків OPC UA, MQTT і Kafka та запустіть як безперервний тіньово-рекомендаційний наглядовий worker, що ніколи не торкається детермінованого керування.

Документ

Продукт

Споживає

Режим безпеки

Платформи

Автор

Дата

Ліцензія

Посібник з промислового розгортання

Jneopallium Industrial Loop Guardian (демо FMI-скіда)

Готову до розгортання мережу з Посібника з навчання

ADVISORY (наглядовий, лише рекомендації)

Windows (PowerShell) і Linux/macOS (bash)

Дмитро Раковський — Харків, Україна

23 червня 2026

BSD 3-Clause

Зміст

1. Чого ви досягнете
2. Передумови — навчений пакет і середовище виконання
3. Архітектура запуску
4. Готовий до розгортання пакет (що створило навчання)
5. Спрямуйте worker на згенеровану структуру нейромережі
6. Спакуйте JAR для розгортання
7. Конфігурація середовища: production-context.json
8. Налаштуйте джерела подій і потоковий вхід
9. Під'єднайте рекомендаційний вивід та запустіть worker
10. Межа керування й безпеки в промислі
11. Перехід: тінь → рекомендації → обмежена автономність
12. Моніторинг у промислі
13. Безпека, відкат і пакування
14. Усунення несправностей
15. Додаток — шпаргалка розгортання

1 Чого ви досягнете

До кінця цього посібника ви візьмете **готову до розгортання мережу, створену Посібником з навчання** — теку конфігурації рівнів/нейронів JNeopallium плюс готовий `production-context.json` — і запустите її як безперервний **наглядовий рекомендаційний worker**, що діагностує знос насоса, коливання, дрейф датчиків і погіршення енергії та рекомендує, але ніколи автономно не керує.

Стеля безпеки для всього посібника

Усе нижче працює в режимі ADVISORY, над PLC/PID/SIS. Worker може рекомендувати обслуговування, обмежені трима уставок та енергетичні дії; він не повинен керувати. Записи виконавчих механізмів OPC UA лишаються заблокованими, доки офлайн-відтворення, тінювий пілот, рекомендаційна підписка та обґрунтування безпеки майданчика не пройдено.

2 Передумови — навчений пакет і середовище виконання

- **Навчена мережа** з Посібника з навчання — тека артефактів із `model-descriptor.json`, `layer-0.json` ... `result-layer.json`, `trained-industrial-loop-guardian-model.json` та `production-context.json`.
- **Java 17+** і зібраний worker (`worker-jar-with-dependencies.jar`).
- **Ваша телеметрія заводу** — теги процесів OPC UA, телеметрія IIoT MQTT і за бажанням тінювий потік Kafka та контекст CMMS / лічильника енергії.
- **Призначення виводу** — приймач JSONL для тінювого режиму або Kafka / SIEM / CMMS / вебхук для видимих оператору рекомендацій.

Спершу навчання

Якщо у вас ще немає навченої мережі, виконайте супровідний Посібник з навчання. Розгортання споживає його вихідну теку напряду, включно з вбудованим `production-context.json`.

3 Архітектура запуску

Промислове розгортання використовує справжній вхідний пункт worker із чотирма аргументами:

```
java com.rakovpublic.jneopallium.worker.application.Entry \  
  <mode> <model-jar-url> <context-class> <context-json-or-path>
```

Аргумент	Значення	Приклад
mode	Топологія розгортання	local, http, grpc-client, grpc-master
model-jar-url	URL вашого JAR для розгортання	file:///opt/jneopallium/industrial-loop-guardian.jar
context-class	Реалізація IContext	...runtime.DemoJsonContext
context-json-or-path	Вбудований JSON або шлях до файлу JSON	.../production-context.json

Entry десеріалізує контекст класом контексту, завантажує JAR моделі в ізолюваний завантажувач класів, потім відображає local, http чи grpc на LocalApplication, HttpClusterApplication чи шлях сервера gRPC — ті самі топології, що в архітектурі.

4 Готовий до розгортання пакет (що створило навчання)

Тренер видає **власне мережу JNeopallium**, а не абстрактну модель. П'ять файлів рівнів плюс опис — це те, з чого завантажується worker:

Артефакт	Роль під час розгортання
model-descriptor.json	Карта всієї мережі: 5 рівнів, 17 нейронів, мапа частот, класи часу виконання
layer-0.json	Межа входу заводу + наглядного контексту
layer-1-fast-telemetry.json	Валідація швидкої телеметрії і стан контуру (7 нейронів)
layer-2-maintenance-energy.json	Чотири навчені діагностичні голови висновків
layer-3-advisory-planning.json	Економічне планування + фіксований запобіжник (4 нейрони)
result-layer.json	Вивід промислових рекомендацій JSONL (2 нейрони)
production-context.json	Готовий контекст рекомендаційного worker

Уся мережа: **5 рівнів, 17 справжніх нейронів, 156 навчених ваг, 4 зсуви**. Кожен нейрон називає конкретний клас із промислового пакета worker — розгортання спрямовує worker на цю теку, а не реалізує щось наново.

5 Спрямуйте worker на згенеровану структуру нейромережі

Структуру нейромережі **генерує навчання**, а не пишуть вручну. configuration.input.layermeta (заданий у production-context.json як model/industrial-loop-guardian/) вказує на теку з п'ятьма файлами рівнів, і worker завантажує мережу напряму. Кожен файл — справжній рівень справжніх нейронів:

Файл рівня	Справжні класи нейронів / роль
layer-1-fast-telemetry.json	SensorNeuron, MeasurementValidatorNeuron, OscillationMonitorNeuron, ...
layer-2-maintenance-energy.json	DegradationModelNeuron, EnergyAccountingNeuron — чотири голови
layer-3-advisory-planning.json	MaintenanceSchedulingNeuron, SetpointOptimiserNeuron, SafetyGateNeuron
result-layer.json	Нейрони виводу рекомендаційного JSONL

Діагностичні голови вбудовують усе, що вивчила модель, готове до запуску — навчені ваги, дозволений кожний featureGate, її logicalNeuronRole та ownedReasoning — тож рецензент підтверджує, що логіка живе всередині моделі, а не в сполучному коді.

Без кроку перекладу

Оскільки структуру згенеровано з уже вбудованими навченими головами, те, що ви розгортаєте, — це саме те, що навчили й протестували: ті самі ваги, ті самі фільтри ознак, ті самі смуги відповіді, той самий фіксований запобіжник.

6 Спакуйте JAR для розгортання

Класи нейронів, процесорів і сигналів, на які посилається структура, **вже постачаються в промислового модулі worker**, тож ви їх не реалізуєте наново. Спакуйте для розгортання:

Компонент	Вміст
Типізовані сигнали	MeasurementSignal, AlarmSignal, DegradationSignal, EfficiencySignal, MaintenanceWindowSignal, SetpointSignal, InterlockSignal
Нейрони	SensorNeuron, MeasurementValidatorNeuron, OscillationMonitorNeuron, DegradationModelNeuron, MaintenanceSchedulingNeuron, EnergyAccountingNeuron, SetpointOptimiserNeuron, SafetyGateNeuron
Процесори	валідація, коливання/заклинювання, планування деградації, планування вікон обслуговування, оптимізація ефективності, запобіжник, диспетчеризація
Джерела	вимірювання/тривоги OPC UA, телеметрія MQTT, тіньовий Kafka, CMMS, контекст лічильника енергії
Приймачі	приймач аудиту JSONL, адаптер Kafka/SIEM/вебхук; приймач команд OPC UA лише з обґрунтуванням безпеки

Перевірка класів із швидкою помилкою

`production-context.json` задає `configuration.neuronnet.classes` повним переліком класів часу виконання. Під час запуску `LocalApplication` перевіряє наявність кожного в JAR і переривається, якщо пакет неповний.

7 Конфігурація середовища: `production-context.json`

Тренер генерує готовий контекст. Він має стандартну форму `{"properties": { ... }}` і вже кодує налаштування рекомендаційного циклу. Критичні ключі:

Ключ	Промислове значення
<code>configuration.input.layermeta</code>	<code>model/industrial-loop-guardian/</code> (згенерована тека рівнів)

configuration.isteacherstudying	false
configuration.discriminatorsAmount	0
configuration.infiniteRun	true
configuration.runoncein	1 мс (один рекомендаційний такт на мілісекунду)
configuration.processing.frequency.map	швидкі теги щоцикл; деградація/енергія кожні 10; обслуговування кожні 60
configuration.outputAggregator	JsonlResultAggregator (замініть на ваш приймач SIEM/Kafka)
industrial.autonomousAction	false
industrial.opcua.role	bounded-local-actuator-path
industrial.mqtt.role	telemetry-and-advisory-only
industrial.neuronOwnedLogic	діагностика, економічний базис, межа безпеки, обмежена рекомендація

Безперервний рекомендаційний цикл

Разом `isteacherstudying=false`, `discriminatorsAmount=0` та `infiniteRun=true` переводять worker у безперервний рекомендаційний цикл. `runoncein=1` задає темп одного такту на мілісекунду, перш ніж мапа частот фанить швидкі й повільні сигнали за їхніми кадансами.

8 Налаштуйте джерела подій і потоковий вхід

Локальне відтворення через Entry використовує вбудований `IndustrialLoopGuardianReplayInput` (запускається `IndustrialLoopGuardianEntryLauncher`). Для промислу під'єднайте специфічний для майданчика `IInitInput` або один із наявних вхідних мостів — вимірювань/подій **OPC UA**, **MQTT**, **FMI** та **PLC4X** — що перетворюють ваші записи на типізовані промислові сигнали, зберігаючи час події. Рекомендовані групи тем Kafka:

Група тем	Відображається на сигнал
plant.telemetry.measurements	MeasurementSignal
plant.telemetry.alarms	AlarmSignal
plant.maintenance.events	DegradationSignal / контекст обслуговування
plant.energy.meters	EfficiencySignal
plant.cmms.workorders	MaintenanceWindowSignal / історія обслуговування
plant.condition.waveforms	MachineWaveformSignal (акустичні / вібраційні потоки)

Тримайте каданс потоку узгодженим із `configuration.processing.frequency.map`: швидкі теги щоцикл; висновки з обслуговування й енергії кожні 10–60 циклів; ознаки й рекомендації здоров'я машини — на власному повільнішому кадансі. Оскільки кореляція ведеться за часом події, запізнiла чи невпорядкована телеметрія все одно потрапляє в правильне місце.

Вхід моніторингу стану машини

Акустичні й вібраційні хвилі надходять як `MachineWaveformSignal`; рівень здоров'я машини на Java видобуває ознаки, оцінює робочий режим і зсув домену, формує гіпотези несправностей і видає рекомендацію — усе лише для читання. Подавайте його з тих самих мостів OPC UA / MQTT / FMI / PLC4X або з виділеного високочастотного потоку сенсорів.

9 Під'єднайте рекомендаційний вивід та запустіть worker

`configuration.outputAggregator` вказує `IOutputAggregator`, що викликається щотижнево із рекомендаціями мережі. Вбудований `JsonlResultAggregator` записує рекомендаційний JSONL — ідеально для приймача аудиту тіньового режиму. Кожна рекомендація несе повне міркування й економічний базис:

```
{
  "asset": "P-101",
  "finding": "pump wear and cavitation risk",
  "confidence": 0.73,
  "evidence": { "neuron": "PumpHealthAndEfficiencyNeuron", "maxVibrationRms": 4.3 },
  "recommendation": "SCHEDULE_PUMP_INSPECTION",
  "recommendedAction": "Inspect P-101 impeller and suction strainer.",
  "urgencyHours": 48,
  "economicBasis": { "neuron": "EconomicBasisNeuron",
    "estimatedAvoidedShutdownValueUsd": 20000,
    "safetyEnvelopeSatisfied": true },
  "controlBoundary": { "plcPidSis": "deterministic control and hard safety remain authoritative",
    "jneopallium": "supervisory diagnosis and bounded recommendations"
  },
  "autonomousAction": false
}
```

Підсистема здоров'я машини видає паралельний `MachineHealthAdvisorySignal` із власною каліброваною формою з урахуванням невпевненості — також лише рекомендаційний:

```
{
  "asset": "P-101",
  "healthScore": 0.62,
  "anomalyProbability": 0.71,
  "faultProbabilities": { "bearingDamage": 0.66, "cavitation": 0.21,
    "imbalance": 0.08, "sensorFault": 0.05 },
  "unknownAnomalyProbability": 0.12,
  "domainShiftScore": 0.18,
  "uncertainty": 0.24,
  "evidence": { "neuron": "MachineHealthCorrelationNeuron",
    "acousticRms": 0.41, "vibrationEnvelopeEnergy": 0.58 },
  "autonomousAction": false
}
```

Для промислу пересилайте цей JSONL у вашу SIEM, тему Kafka, CMMS чи вебхук. Потім запустіть worker (роздільник classpath: ; у Windows, : у Linux/macOS):

```
java -cp worker/target/worker-jar-with-dependencies.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local \
  file:///opt/jneopallium/industrial-loop-guardian.jar \
```

```
com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
/opt/jneuropallium/model/industrial-loop-guardian/production-context.json
```

10 Межа керування й безпеки в промислі

Межа, яку забезпечує демо, — це межа, яку ви тримаєте в промислі. Саме вона робить продукт страхованим і легким для валідації:

- **PLC / PID / SIS** володіють детермінованим мілісекундним керуванням і жорсткою безпекою. Завжди авторитетні.
- **OPC UA** — єдиний обмежений шлях виконавчих механізмів, і лише коли обґрунтування безпеки дозволяє записи. Три командні вузли; кожен запис проходить ланцюг пріоритету нижче.
- **MQTT** — лише телеметрія й рекомендації; міст структурно відхиляє AUTONOMOUS.
- **Здоров'я машини** — лише для читання: MachineHealthAdvisorySignal ніколи не записує команду виконавчого механізму; інтерлоки, запобіжник і перевизначення оператора лишаються авторитетними й над ним.

```
жорсткий інтерлок -> локальний fail-safe -> перевизначення оператора -> режим безпеки
-> валідація/якість -> обмеження -> ліміт рампи -> придушення різниці
-> запис OPC UA -> аудит
```

Безпечні значення при втраті: клапан охолодження 100% відкрито, швидкість насоса 30%, потужність нагрівача 0%. При втраті OPC UA скід застосовує їх локально; при втраті MQTT доступність швидкого контуру лишається 1.0.

11 Перехід: тінь → рекомендації → обмежена автономність

Переходьте поступово. Кожен крок заслуговує наступний; записи OPC UA лишаються заблокованими, доки вся послідовність не пройдена.

1. **Офлайн-відтворення.** Аналізуйте експортовані дані історіана / OPC UA / MQTT без підключення до промислу. Видайте виявлені аномалії, хронологію подій, оцінений фінансовий вплив і перегляд хибних спрацювань.
2. **Тіньовий пілот.** Підключення до заводу лише для читання, без записів виконавчих механізмів. Порівняйте рекомендації з фактичними подальшими результатами й базовою лінією простого алгоритму.
3. **Промислова рекомендаційна підписка.** Видимі оператору рекомендації з економічним базисом, коли досягнуто бюджетів хибних спрацювань і цілей затримки.
4. **Обмежена автономна оптимізація.** Лише після суттєвих тіньових доказів і обґрунтування безпеки майданчика — вузько обмежені рекомендації чи команди уставок, що ніколи не замінюють інтерлоки PLC чи сертифіковану SIS.

12 Моніторинг у промислі

- Вимірювання, оброблені на джерело за хвилину; відкинуті, запізнані, невпорядковані події.
- Кількість рекомендацій на актив; рівень перегляду хибних; підтверджений рівень істинних позитивів.
- Час випередження виявлення (наскільки раніше за можливу відмову чи тривогу).

- Оцінки енергії та простою проти базової лінії простого алгоритму.
- Версія моделі та `trainingChecksum` для кожної рекомендації; події інтерлоків і перевизначень.

Кожна рекомендація зберігає родовід: актив, сигнали, що долучилися, нейрон-власник, економічний базис, результат межі безпеки та твердження про межу керування.

13 Безпека, відкат і пакування

Пакування

- Згенерована тека артефактів (файли рівнів, модель, опис, `production-context.json`).
- SHA коміту Git, команда навчання, `trainingChecksum`, контрольні суми згенерованих артефактів, запис погодження рекомендаційного розгортання та ідентифікатор пакета відкату.

Правила безпеки та відкату

- Жорсткі запобіжники — фіксована конфігурація в `layer-3-advisory-planning.json`, ніколи не навчені ваги; інтерлоки й перевизначення оператора лишаються авторитетними.
- Тримайте попередній пакет мережі доступним для негайного відкату.
- Відкочуйте, якщо хибні спрацювання перевищують бюджет, покриття джерел падає або базову лінію простого алгоритму більше не перевершено.

Чесне обмеження

Вбудована еталонна мережа навчена на синтетичному детермінованому корпусі скіда. Вона чудова для відтворюваних доказів конвеєра та регресії межі безпеки, але це доказ симуляції/HIL — не сертифікований контролер безпеки. Вона не замінює валідацію PLC/SIS, аналіз небезпек, керування змінами чи приймальні випробування на майданчику. Реальна точність потребує валідації на зовнішніх даних історіана, CMMS та лічильників енергії.

14 Усунення несправностей

Симптом	Імовірна причина й виправлення
Cannot find class ... in provided jar	Бракує класу з <code>configuration.neuronnet.classes</code> — перезберіть JAR розгортання
Worker стартує, але нічого не видає	Перевірте <code>clazz</code> джерела подій і що <code>readSignals()</code> повертає типізовані сигнали
Структуру не завантажено	Переконайтеся, що <code>configuration.input.layermeta</code> вказує на згенеровану теку рівнів
Worker одразу завершується	Для живого наглядача встановіть <code>infiniteRun=true</code> ; <code>maxRun</code> — для обмеженого відтворення
Рекомендація намагається керувати	Перевірте <code>industrial.autonomousAction=false</code> і що рівень відповіді ADVISORY
Застарілі ваги під час роботи	Ви розгорнули стару теку рівнів; розгорніть найновіший навчений пакет

15 Додаток — шпаргалка розгортання

Запуск промислового рекомендаційного worker

```
java -cp worker/target/worker-jar-with-dependencies.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local file:///opt/jneopallium/industrial-loop-guardian.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  /opt/jneopallium/model/industrial-loop-guardian/production-context.json
```

Основи контексту рекомендаційного worker

```
configuration.input.layermeta      = model/industrial-loop-guardian/
configuration.isteacherstudying    = false
configuration.discriminatorsAmount = 0
configuration.infiniteRun          = true
configuration.runoncein            = 1
industrial.autonomousAction        = false
```

Jneopallium Industrial Loop Guardian · Посібник з розгортання. Споживає мережу, створену Посібником з навчання. Наглядовий над PLC/PID/SIS. Режим безпеки: ADVISORY. Записи OPC UA потребують окремого обґрунтування безпеки майданчика. Ліцензія: BSD 3-Clause.